

Lab 3 Component Modelling and Architecture

Name:- Prithvish Adhikari

SRN:- PES1UG24AM206

Group Travel Expense Simplification Engine

Architecture justification

We chose Layered Architecture for the Group Travel Expense Simplification Engine, implemented as a modular backend with a web client. The presentation layer serves trip admins and travelers, the business layer validates expenses and computes settlements, and the data layer maintains a shared transactional ledger.

Scenario and challenges: The system creates trips, manages participants, records equal or uneven expense splits, converts currencies, calculates net balances, and tracks settlement payments. Its main challenges are accurate rounding, concurrent updates, trip-level privacy, exchange-rate failures, and understandable settlement instructions.

Style comparison: Layered architecture separates these concerns and supports local transactions, but individual modules cannot scale independently. Microservices allow independent deployment and scaling, but distributed transactions and operational overhead are unnecessary for this lab. Client-server supports a shared ledger and centralized control, but alone does not define the internal separation of responsibilities. Our layered design also uses a client-server deployment.

Reason 1 for the choice: Expenses, participant shares, and settlement records must agree. A common repository allows atomic updates and consistent reads, reducing the risk of incomplete expense splits or conflicting payment records.

Reason 2 for the choice: The five project epics map to focused modules with explicit interfaces. Currency lookup can change without changing expense entry, while settlement logic can be tested independently of the user interface.

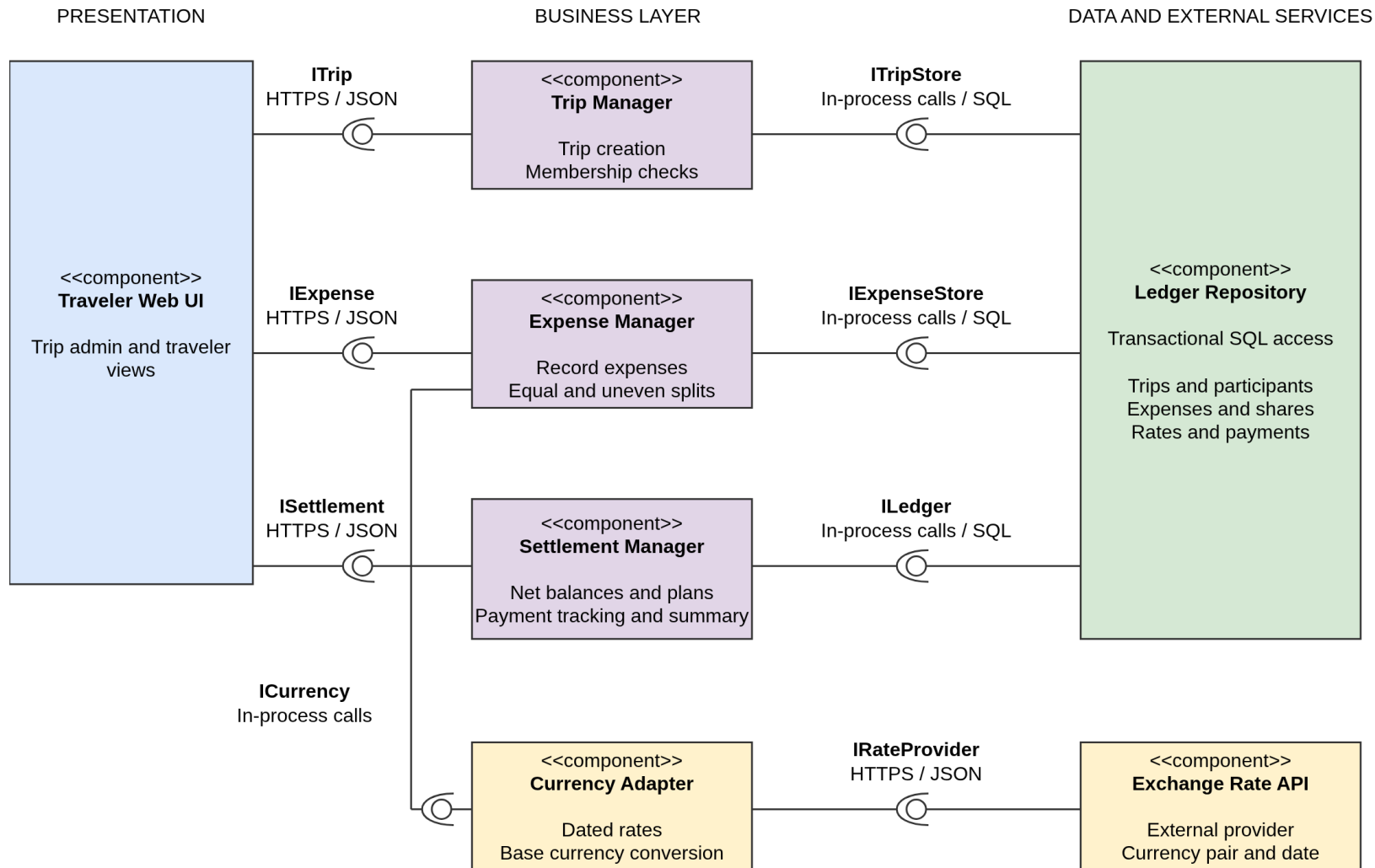
Security advantage: Business endpoints authenticate users and verify trip membership and admin permissions before reading or changing records. The client cannot access SQL directly; parameterized repository queries and server-held exchange API credentials reduce exposure.

Performance benefit: Business modules communicate in-process, avoiding service-to-service network calls for ordinary ledger operations. Batch reads and indexes on trip identifiers reduce repeated queries. Cache rates by currency pair and effective date, and retain the rate used with each expense for auditability.

Drawbacks: The backend and database remain shared failure and scaling points. Settlement search also needs a defined maximum group size: an exact minimum-transfer search can be expensive, and a greedy fallback must not be described as a guaranteed minimum.

UML component diagram

Group Travel Expense Simplification Engine



Circle = provided interface Socket = required interface Solid line = assembly connector

Requests travel from the socket component to the circle component; results return over the same connection.

Created and exported in draw.io. Six internal components and one external provider; eight named interfaces. Line crossings without a junction are not connections.

Components and interface contracts

The diagram is a proposed design for the existing project. Interfaces show contracts rather than separate deployments. The Ledger Repository encapsulates persistence and its SQL database; the Exchange Rate API is outside the application boundary.

Component	Responsibility and epic mapping
Traveler Web UI	Trip, expense and settlement screens; supports all five epics.
Trip Manager	Trip creation and membership authorization; Epic 2.
Expense Manager	Expense validation and equal or uneven shares; Epic 1.
Currency Adapter	Dated rate retrieval and conversion; Epic 3.
Settlement Manager	Net balances, optimized plans, summaries and payment recording; Epics 4 and 5.
Ledger Repository	Persist trips, participants, expenses, shares, rates and payments.
Exchange Rate API	External rate source queried by currency pair and date.

Interface consumers and providers

Interface	Consumer to provider	Request and returned result
ITrip	Web UI to Trip Manager	Create trip or change participants; trip state.
IExpense	Web UI to Expense Manager	Record expense and shares; expense ID or validation error.
ISettlement	Web UI to Settlement Manager	Read balances or record payment; plan, summary or receipt.
ITripStore	Trip Manager to Repository	Store or read trip membership; authorized trip data.
IExpenseStore	Expense Manager to Repository	Commit expense, shares and rate atomically; saved record.
ILedger	Settlement Manager to Repository	Read a consistent ledger or record payment; updated balances.
ICurrency	Expense Manager to Currency Adapter	Amount, currency pair and date; converted amount and rate.
IRateProvider	Currency Adapter to Exchange Rate API	Pair and date; rate and effective date or lookup error.

Example interaction. A traveler submits an expense through IExpense. After authorization and split validation, Expense Manager calls ICurrency when conversion is required and saves the complete record through IExpenseStore. Settlement Manager reads ILedger to calculate what each traveler owes. A failed rate lookup blocks conversion with a clear error instead of silently using an invalid rate.